

Chapter 7. Tips and Tricks for Deep Learning in Biology

This final chapter brings together common themes from earlier chapters and distills practical strategies for applying deep learning techniques to biological problems. In machine learning, it's rare for things to work perfectly on the first try—or even the tenth. Debugging is an expected part of the process, not a sign of failure. Don't get discouraged.

Here, we share a collection of tips that have helped us (and others) navigate the challenges of deep learning in biology. Some were learned the hard way, and others emerged from writing this book. This list isn't exhaustive, but we hope it shortens your path to developing working models—and sharpens your instincts for when things go wrong.

WARNING

Don't expect steady, incremental improvements in your project. Progress in deep learning—especially with biological data—is often highly nonlinear. You might spend weeks debugging with no clear gains, only to make one small change that suddenly unlocks everything. This is normal—and not a cause for concern.

Simplify

When things stop making sense, simplify. Strip your problem back to the bare minimum—a smaller dataset, a shallower model, or a simpler loss function. It's easy to get lost in complex pipelines, but debugging is much easier when you can isolate one thing at a time.

Once you’ve got something working again, you can reintroduce complexity gradually. Think of this as turning the knobs one by one instead of all at once. It’s not glamorous, but it’s one of the most reliable strategies for making progress.

Simplify Your Model

When your model has too many bells and whistles, it becomes difficult to pinpoint where things are going wrong. Often, the most effective debugging strategy is to strip it down to the bare essentials.

Simplify your architecture

Complex architectures can make it difficult to reason about what’s going wrong. To help with this:

Eliminate unnecessary layers

If a layer doesn’t contribute directly to the input–output mapping—such as layers that simply add model capacity without altering dimensions—it’s best to remove it during debugging.

Call basic layers directly

Use layers like `nn.Conv` and `nn.Dense` instead of custom blocks, which can obscure bugs and internal behavior.

Reduce depth and width

If your model has many layers or units per layer, consider reducing both. A shallower model is easier to debug and understand, especially in the early stages of development.

Remove residual connections

These can complicate debugging by introducing dependencies between layers and by masking issues in the layers they connect (like poor initialization or gradient problems).

TIP

The ultimate simplification is to reduce your model to a direct mapping from inputs to outputs. For example, pass your input through a single `nn.Dense` layer and see if it can overfit a tiny batch of data. If that fails, the issue is likely not with the architecture, but with your data pipeline, loss function, or optimizer.

Turn off extras, like normalization and dropout

These add complexity that's often unnecessary during early debugging. In Flax, you must explicitly manage both model state (e.g., batch norm statistics) and random number generators (RNGs), which can easily lead to subtle bugs.

Don't use batch norm

The complexity of batch norm is threefold. First, it behaves differently during training and inference. Second, it introduces additional state (running mean and variance) that must be updated outside standard gradient updates. Third, it breaks a key assumption: most layers operate independently on each batch element, but batch norm computes statistics across the batch. This makes it incompatible with tools like `vmap` or sharded training (`pmap`, `pjit`) unless you take special care to synchronize statistics across devices.

Skip dropout

Dropout introduces stochasticity, making it harder to determine whether poor performance is due to randomness or a deeper issue.

Simplify your optimizer

Don't worry about experimenting with different optimizers or learning rate schedules until the basics are working. Pick a sensible default—like good old Adam with a learning rate of `1e-3`, and focus on solving more fundamental issues first.

Avoid mixed precision

While lower-precision data types like `bfloat16`, `float16`, or `TensorFloat32` can improve performance and memory usage (out of scope for this book), they can lead to subtle numerical instability

that's extremely hard to debug. Note that even if you pass in `float32` inputs, JAX may default to lower-precision matmuls. To force full `float32` precision globally (especially during debugging), add

```
jax.config.update("jax_default_matmul_precision",  
"float32").
```

NOTE

Although turning off these more advanced features may lead to worse performance (due to underfitting or overfitting) and feel like you're going backward, once the basic setup is functioning correctly, you can systematically reenable features to assess their impact on model performance.

Simplify and Control Your Environment

Your model might be fine, but your technical environment could be introducing unexpected issues. Many bugs that look like deep learning failures are really just environmental gremlins—this means that cleaning up your setup is often the fastest way forward. Here are some tips to keep in mind:

Sort out determinism and reproducibility

It's easier to isolate issues if your experiments are reproducible. In addition to turning off stochastic components like dropout, consider:

Setting explicit random seeds

For reproducibility in JAX, you need to set the seed for `jax.random.PRNGKey(...)`—this controls all randomness in model initialization, dropout, and other JAX-based operations. Note that you don't need to set Python's `random.seed(...)` or NumPy's `np.random.seed(...)` unless you're also using them separately (e.g., in your data preprocessing or non-JAX components).

Turning off dataset shuffling

Don't shuffle your training data, or shuffle with a fixed random seed, to maintain a consistent order of examples across runs.

Keeping the environment constant

Avoid inconsistencies caused by external factors (e.g., use the same hardware, library versions, and configurations).

Strip down your training loop

Especially in JAX and Flax, training loops require manual control over RNGs, state, and updates—making them powerful but easy to get wrong.

Train on a single batch for just a few steps

This is often enough to catch major issues.

Disable extras like logging, metrics, or learning rate schedules

These can obscure what's actually happening during training.

Use fixed inputs and random seeds

Run your training step on the same batch every time. This eliminates variability due to changing data and makes bugs easier to isolate.

Avoid high-level abstractions temporarily

If you're using tools like `TrainState`, try replacing them with raw variable updates until the core logic works.

Make your code self-contained

Especially when working in interactive environments like Colab or Jupyter notebooks, it's easy for your environment to get cluttered with old variables or states without you realizing it. Restarting the kernel and organizing your code into self-contained functions can really help with debugging.

Turn off JIT compilation (with caution)

Disabling `@jax.jit` can help identify issues in your training step more easily by making stack traces clearer and behavior more explicit. However, be aware: turning off JIT can cause huge

memory usage spikes and drastically slower execution, making this approach impractical for larger runs.

Train on a single GPU instead of multiple GPUs

If you are using GPUs, start with just one. Multi-GPU training via sharding (e.g., with `jax.pjit`) introduces a lot of additional complexity. (Note: `pmap` is being deprecated in favor of `pjit`, so it's best not to rely on it.)

Use CPU for simpler debugging setups

If your model is small and you're trying to isolate a bug, using CPU can remove the complexity of device placement and driver issues. It's slower, but often easier to reason about. Just be aware that some bugs may appear only on accelerators.

Go back to NumPy

While `jax.numpy` mimics NumPy closely, the original NumPy library has simpler internals and often better error messages. You can't train models or use autodiff, but for testing data transformations or verifying calculations, it can be useful to isolate and debug numpy-only code outside of JAX.

These steps may feel tedious, but a clean and controlled environment is often the difference between spinning your wheels for days and finding a bug in 10 minutes.

A quick note: different hardware backends behave slightly differently when running JAX code, especially when it comes to reproducibility.

- *TPUs* are fully deterministic in JAX. If you use the same random seed, you'll get the same results every time.
- *GPUs* are mostly deterministic, but some operations (like convolutions or matrix multiplications) may vary slightly across runs unless you disable certain performance optimizations.
- *CPUs* are generally deterministic, but subtle sources of non-determinism (like thread scheduling) can still appear.

For early debugging in JAX, running on CPU can simplify things—but just be aware that bugs might show up differently when you move to GPU or TPU.

Simplify the Data and Problem

Make your dataset and prediction task easier to simplify debugging. Here are some ways to make the data more manageable during early experimentation:

Visualize individual examples

Actually plot or print raw inputs and labels—not just summaries. You'll often catch issues like incorrect encodings, off-by-one errors, or mismatched image-label pairs this way. It's surprisingly tempting to skip this step—don't. Simply looking at the raw data can reveal issues that save you hours later.

Check class balance

An imbalanced dataset can make the model appear broken when it's just doing the naive thing (predicting the majority class). Consider subsampling or rebalancing during debugging.

Remove data augmentation

Augmentations like cropping, flipping, or adding noise can hide underlying issues or make the task unnecessarily hard. Turn them off until you're confident the core pipeline works.

Reduce the number of classes

Instead of predicting many categories, reframe your task as binary classification to focus on the clearest signal first.

Simplify the output space

On a similar note, if your target is complex (e.g., a regression label or structured output), try reducing it to something simpler. For instance, predict a binary class or thresholded version of the label instead. This can help verify the pipeline before tackling the full problem.

Make your dataset smaller

Large datasets slow everything down. Use a small, representative subset that captures the key structure.

Limit the scope of your data

Use a natural slice of your dataset. For example, restrict to a single species, tissue type, year, or patient group. This cuts variability and helps isolate bugs.

Check for label leakage

Especially in biological datasets, label leakage can creep in through metadata like patient ID, batch number, or experiment date. This can cause your model to perform suspiciously well by learning shortcuts. Double-check that no features or splits accidentally leak target-related information.

Work with synthetic or simulated data

If feasible, start with synthetic data that mimics key characteristics of your real dataset but is easier to understand and trace. You can also add controlled noise (e.g., `np.random.normal(0, 1)`) to test the model's robustness.

Simplifying the data isn't about giving up—it's about creating a controlled testbed where you can debug quickly, eliminate uncertainty, and build confidence before scaling back up.

Overfit to a Single Batch of Data

We've mentioned this briefly before, but it's worth stating explicitly: if your model doesn't seem to be learning much at all, a classic debugging strategy is to try to overfit to a single batch—meaning, rerun the training

loop on the same batch repeatedly and see if the model can memorize it. This test helps confirm that the training loop, loss function, and optimizer are wired up correctly.

Instead of the usual training loop:

```
for step in range(num_steps):  
    batch = next(training_data)  
    state, loss = update_step(state, batch)
```

you can try this:

```
num_debugging_steps = 10  
batch = next(training_data)  
  
for step in range(num_debugging_steps):  
    state, loss = update_step(state, batch)  
    print(f"Loss at step {step}: {loss}")
```

If your setup is working correctly, the model should rapidly memorize the batch, and the loss should decrease significantly after a few steps. If not, consider checking for:

Learning rate issues:

The learning rate might be too high (causing divergence) or too low (causing no learning).

Frozen parameters or bad gradients

Sometimes parameters are not being updated at all—for example, if they were accidentally excluded from the `params` dict due to naming mismatches or scoping issues. Also inspect gradients—if they're all zero or NaN, that's a clue.

Loss function bugs

Make sure you're using the correct loss function for your task (e.g., cross-entropy for classification) and that it's behaving numerically

as expected. Prefer standard implementations over custom home-made ones, at least during debugging.

Model initialization problems

Poor or inconsistent weight initialization can prevent learning, especially in deeper networks. If you're using custom modules, double-check their initializations.

Batch size too small

Very small batches (e.g., 1–2 examples) can lead to noisy gradients and unstable updates. For debugging, use a small but reasonable batch size like 8–32.

Silent shape mismatches or broadcasting errors

These won't always crash your code, but they can silently mess up your loss or gradients. Print tensor shapes and inspect intermediate outputs to confirm everything lines up as expected.

This is one of the fastest and most informative debugging steps—if your model can't learn a tiny batch, don't bother scaling up yet.

Go Back to Basics

If none of these debugging tips works and you are rapidly losing your mind, one of the most effective strategies is to revert to a simple, well-understood example that you know works:

Start with an established example

Train a simple model on a well-known dataset—like a basic CNN on MNIST for image problems. These examples are widely used and well-documented, making them a reliable way to get something working end-to-end.

Reproduce known results

Make sure your setup can successfully train the model and reach the expected performance (e.g., ~99% accuracy on MNIST). This confirms your training loop, model, and loss function are functioning correctly.

Swap in your dataset

Once the baseline works, begin replacing the dataset with your own. Proceed gradually and check that everything still functions as expected.

Iteratively add complexity

With your data integrated, incrementally introduce more complex components—like deeper architectures or new training strategies. Watch for breakage after each change.

There's absolutely nothing wrong with going back to a tutorial and training a simple linear model—sometimes that's the fastest way to confirm your setup and get your bearings.

Log Everything

Good logging is the difference between productive debugging and blindly guessing. When something goes wrong, clear logs can help you trace back exactly what happened—and when things go right, they help you understand why.

Log training loss and key metrics over time

At a minimum, track loss, accuracy, and any relevant task-specific metrics (like auROC or auPRC). This makes it easier to spot overfitting, instability, or underperformance.

Log validation performance at regular intervals

Seeing how your model generalizes during training helps detect overfitting early and can catch bugs where validation performance diverges for no obvious reason.

Log inputs, predictions, and errors

Save a few input samples, predicted outputs, and errors at each step (or epoch). This is especially useful for spotting systematic failures (e.g., always misclassifying a certain class).

Record configuration and hyperparameters

Save the learning rate, batch size, optimizer type, and model architecture along with each run. You will forget. Everyone forgets.

Use a structured logger or tracking tool

Tools like TensorBoard, Weights & Biases, or even just structured JSON logs can make it easier to compare runs and understand what changed.

TIP

Logging might feel like overhead in the moment, but it's one of the best time investments you can make. Even minimal logs can help you debug faster and avoid retraining unnecessarily.

There's a rule of thumb: every new log reveals a bug you didn't know you had.

Ask for Help

If you're still stuck, don't be afraid to reach out. Community forums like Stack Overflow or GitHub Discussions are valuable resources. Talking to a colleague or friend can also help—sometimes just explaining the problem out loud (rubber ducking) leads to breakthroughs.

You can also use LLMs like ChatGPT, Gemini, or Claude to help troubleshoot or explore ideas. Just remember that while these models can be very helpful, their suggestions aren't always correct and can introduce new bugs—so double-check any code they generate.

Common Data Issues

Often, it's not your model that has a bug—it's the data. Subtle issues in your dataset can quietly break your learning pipeline, and you may find yourself spending hours debugging the model setup when the problem is actually upstream. This section covers common data pitfalls that are worth checking before tearing your architecture apart.

Data Leakage

As physicist Richard Feynman famously said, “The first principle is that you must not fool yourself—and you are the easiest person to fool.” It’s all too easy to believe your model is doing well when it’s actually cheating. Data leakage happens when information that should be hidden during training is inadvertently accessible to the model, leading to overly optimistic performance metrics.

- Obvious cases
 - Evaluating the model on the training set itself. This sounds slightly silly, but it’s surprisingly common—especially in informal settings like Kaggle notebooks.
 - Evaluating on a validation or test set where some examples overlap with the training set.
- Subtle cases
 - Leakage through preprocessing, for example, normalizing the full dataset before splitting into train/valid/test sets.
 - Features that leak future information—correlated with the target only because they wouldn’t be available at prediction time.

Here are some real-world examples of subtle leakage:

- You want to classify skin lesions as malignant or benign. Patients with cancer are photographed in one clinic and healthy patients in a different clinic. If one clinic uses brighter lighting, your model may learn to use brightness as a proxy for cancer.
- You’re predicting protein binding to a gene, and you include gene expression level as an input to the model. Since a gene’s expression may be affected by binding, your model might learn to rely on this proxy instead of the DNA sequence features you intended it to learn.

WARNING

If your model performs well on a test set but fails to generalize to new datasets or real-world settings, data leakage is one of the first things to investigate.

To avoid data leakage:

Always ensure that test data is fully isolated and untouched by the training pipeline.

When adding new training data partway through a project, check whether it already appears in your validation or test sets.

Ask yourself: *Would this feature be available at inference time?* If not, don't use it.

Use model interpretation tools to see what aspects of your data your model is relying on when making its predictions. Does what you see match your expectations, or is the model picking up on artifacts?

Incorrect Data Labels

It seems obvious, but incorrectly labeled data is one of the most common and frustrating causes of model underperformance. Some high-risk scenarios include:

Labels stored separately from inputs

If labels are in a separate file (e.g., a CSV with filenames and classes), they can easily be mismatched or misjoined during pre-processing. We certainly have several shameful anecdotes along these lines.

Shuffling inputs and labels independently

If you shuffle data and labels separately, they'll fall out of sync—silently.

Shape mismatches in TensorFlow datasets:

`tf.data.Dataset` won't necessarily complain if your labels and inputs have mismatched shapes like data of shape `(100, 10)` but labels of shape `(43, )`. This can result in silent failures that only manifest much later.

Merging tabular datasets incorrectly

Joining datasets without verifying alignment (e.g., via `merge` in pandas) can mislabel data rows without throwing errors.

Data augmentation pipelines modifying labels incorrectly

Augmentation effectively increases your dataset—so it’s also a high-risk area for introducing label corruption.

NOTE

A common warning sign of label issues: the training loss does goes down slightly during training but plateaus early at a high value, and accuracy (or other metrics) stays near random chance level.

For example, if the labels are scrambled, your accuracy would hover around random baseline values like 50% in balanced binary classification.

The best antidote to label issues is simply spending time inspecting your input-label pairs, both at the beginning as raw data, and at different points in your data pipeline. Check a few batches by hand. Plot examples and verify the label. It may feel tedious—but it’s one of the fastest ways to catch silent bugs.

Imbalanced Classes

In biology, it’s common for one class to vastly outnumber others—like detecting rare mutations or identifying diseased cells. This class imbalance is not an issue in itself, but a model trained on imbalanced data might just learn to always predict the majority class, achieving deceptively high accuracy.

Warning signs:

- Accuracy is high, but precision or recall on the minority class is poor.
- Confusion matrix shows the model rarely predicts the minority class.

To address this:

- Use class weighting or focal loss to penalize the model more for mistakes on the minority class. Focal loss down-weights easy examples and focuses learning on hard, misclassified ones—especially useful when the rare class is easily overwhelmed.
- Resample the data—either oversample the minority class or under-sample the majority class—to reduce imbalance. Oversampling is often safer when data is scarce but can lead to overfitting if not done carefully.
- Use stratified sampling to ensure class balance is preserved across your train, validation, and test splits. This means splitting the data so each subset maintains the original class proportions, avoiding skewed performance estimates.

Distribution Shifts

Training and test data can often come from different sources—different labs, species, sequencing protocols, or imaging setups. These shifts can cause models to learn dataset-specific artifacts instead of generalizable biology.

Warning signs:

- Strong validation performance doesn't transfer to real-world or external datasets.
- A model trained to predict dataset labels (e.g., lab or batch ID) performs surprisingly well.

To catch and correct for this:

- Visualize embeddings (e.g., via PCA or UMAP) colored by data source to spot clustering.
- Use batch effect correction or domain adaptation methods if needed.
- Be cautious when mixing data from different sources—explicitly test generalization to new settings.

Biology-Specific Gotchas

Biology is a vast field, filled with complex systems and ever-evolving datasets. We can't cover every pitfall here, but the following are some common sources of bugs and errors that we've encountered repeatedly—and that are worth keeping in mind when building models on biological data:

Versioning issues

Many biological datasets are tied to reference versions (e.g., genome builds, gene IDs, transcript annotations). It's dangerously easy to mix up genome versions (e.g., GRCh37 vs GRCh38), gene ID versions, or even organism accessions. Make sure all parts of your pipeline are using consistent versions—or explicitly map between them.

Data integration challenges

Combining datasets from different sources is common in biology but can lead to subtle inconsistencies: mismatched identifiers, differing file formats, or incompatible measurement units (e.g., read counts versus TPM versus RPKM). Carefully check alignment before merging datasets.

Biological heterogeneity

Biological systems vary widely—across individuals, cell types, populations, and species. A model trained on European ancestry samples may not generalize to other ancestries. Likewise, models trained on data from immortalized cancer cell lines can fail when applied to normal primary cells. Always consider the scope and limitations of your training data.

Ambiguous or soft labels

Biological categories are often not cleanly defined: cell types can be graded or transitional, and protein binding is often a continuous score, not a binary yes/no. Hard labels, where present, may oversimplify what is actually a spectrum. In these cases, performance ceilings may reflect label ambiguity, not model failure.

Experimental noise

Just adding more data isn't always better—low-quality experimental data can introduce noise that overwhelms signal. Look for ways to filter or denoise.

Use quality metrics

Many experiments include built-in quality scores. Filtering on these can help.

Leverage replicates

Use experimental replicates (same exact setup, multiple times) or biological replicates (same protocol, but different samples) to reduce variance. You can average replicate signals or use them to quantify uncertainty.

Batch effects

Differences in lab conditions, reagent lots, sequencing machines, or protocols can introduce strong confounding signals. These technical artifacts often dominate the true biological signal if not accounted for. Visualize your data (e.g., with PCA or UMAP colored by batch) to assess how much batches cluster apart. You can also train a model to predict batch labels—if it performs better on this than your actual task, your model is probably learning batch-specific noise. If needed, apply normalization techniques like quantile normalization to mitigate these effects.

Common Model Issues

Not all training failures come from bad data—sometimes, the model itself is the problem. In this section, we highlight common issues that arise during model training, from overfitting to gradient instability.

There are more detailed deep learning debugging guides out there—for example, the [Deep Learning Tuning Playbook](#) by Google. But here we'll recap some of the most common and practical failure modes, with a focus on how to identify and fix them efficiently.

Overfitting and Poor Generalization

Overfitting is one of the most common issues in deep learning. Deep neural networks typically have high capacity and are only told to minimize training loss—without any built-in notion of generalization. As a result, they often perform well on training data but poorly on unseen data.

Fortunately, there's a well-established set of regularization techniques to help reduce overfitting—many of which we've already discussed throughout this book:

Dropout

Randomly disables units in the network during training to prevent over-reliance on any one path.

Weight decay

Penalizes large weights (L1 or L2 regularization) to encourage simpler models that generalize better.

Early stopping

Monitors validation performance and stop training when performance starts to degrade, even if training loss continues to drop.

Data augmentation

Expands your dataset by applying small, meaningful transformations (e.g., rotations, flips in images; jittering or cropping in sequences).

Ensembling

Combines predictions from multiple models trained with different seeds or splits as a form of error correction. Even ensembling the same architecture can significantly improve robustness.

Also, check whether your validation or test data differs fundamentally from your training data (see the previous section). If the distribution has genuinely shifted, then the model might not be overfitting so much as encountering data it was never trained to handle.

A model that performs well on training data but poorly on validation is likely overfitting. A model that performs poorly on both might be underfitting or struggling with a broken setup.

Vanishing or Exploding Gradients

Vanishing gradients, where gradient values approach zero, and *exploding gradients*, where they become excessively large, can severely disrupt training. Fortunately, these issues are relatively easy to detect.

A simple way to monitor gradients is to compute their L2 norm (also called the Euclidean norm), which summarizes the overall magnitude of the gradient as a single scalar. You can log this value alongside the loss during training.

Here's how to compute the L2 norm of gradients in Flax:

```
@jax.jit
def compute_gradients_l2_norm(grads):
    """Compute L2 norm of gradients."""
    grads_flat = jax.tree_util.tree_leaves(grads) # Flatten.
    return jnp.sqrt(sum([jnp.sum(jnp.square(g)) for g in grads_flat]))

# Example usage inside a training step:
loss, grads = jax.value_and_grad(loss_fn)(state.params)
grad_norm = compute_gradients_l2_norm(grads)
```

You can log this `grad_norm` over time and visualize it alongside the loss to examine gradient behavior:

- If `grad_norm` is close to 0, gradients are likely vanishing.
- If it grows rapidly or spikes erratically, you may be seeing exploding gradients.

Common fixes to try out include:

- Lower the learning rate or use a learning rate schedule.
- Use better weight initializers: try Xavier (Glorot) or He initialization, depending on your activation function.
- Normalize activations: Batch normalization or layer normalization helps stabilize the flow of gradients.
- Add residual connections: These help gradients propagate through deep networks without degradation.
- Clip gradients: This is a blunt but effective tool to cap extreme values and prevent instability.

The fixes to this issue tend to be ones we've already mentioned, like reducing the learning rate or using a learning schedule, using different weight initializers, and adding either batch normalization or layer normalization. Adding residual connections between blocks can also be helpful. Finally, explicitly clipping gradients to a fixed threshold to avoid excessive values might sound like a really crude approach but is common and very effective. Here is an example implementation:

```
def clip_gradients(grads, threshold):  
    """Clip gradients."""  
    return jax.tree_map(lambda g: jnp.clip(g, -threshold, threshold), grads)
```

Or, if you are using `optax`, you can also clip gradients with:

```
tx = optax.chain(optax.clip(threshold), optax.adam(learning_rate))
```

Training Instability

A related issue to gradient issues is *training instability*, which can manifest in several ways, including erratic training losses, sudden spikes in validation loss, or even full-blown divergence. By *divergence*, we mean that the model fails to *converge* toward a stable solution; instead, the loss may oscillate wildly or become `NaN`.

Training instability typically arises from a few common causes:

Learning rate is too high

A high learning rate can cause the optimizer to overshoot minima, leading to instability. Try lowering the learning rate or using a warmup schedule that starts small and ramps up gradually.

Using a nonadaptive optimizer

Adaptive optimizers like Adam, RMSProp, or Adagrad adjust learning rates per parameter and tend to be more robust out-of-the-box. While vanilla stochastic gradient descent (SGD) can be effective, it typically requires more careful tuning, especially with larger models or noisy data.

Exploding gradients

In deep networks, gradients can grow too large and destabilize updates. As discussed earlier, apply *gradient clipping* or use normalization layers (like batch norm or layer norm) to control this.

Inappropriate batch size

Very small batches can lead to noisy gradient estimates that make training unstable. Larger batches offer more stable gradients—generally, try using the largest size your hardware allows, especially during early debugging.

Poor weight initialization

Improper initialization can cause gradients to vanish or explode. Flax uses LeCun normal as the default initializer for `nn.Dense` and `nn.Conv`, which works well with ReLU activations. But for very deep networks or specific architectures, Xavier or He initialization may perform better.

Activation blowup

As networks deepen, intermediate activations can grow excessively large, especially with ReLUs or unnormalized inputs. To prevent this, keep activations centered and bounded, most commonly by applying batch normalization.

Poor Model Performance

The model trains and the dataset looks good. You've squashed overfitting. Everything generally looks sane. The only problem is that the model is just not that good.

How Well Should You Do?

We touched on this point in the introduction, but it's worth restating: to judge performance, you need context. Here are some ways to anchor your expectations:

Random chance performance

What would random guessing achieve? For regression, how well would you do by always predicting the mean or median of the training set?

Baseline models

Try a simple linear regression or logistic regression. Sometimes these models perform surprisingly well—and if your deep model doesn't beat them, something's wrong.

Other published models

If others have worked on this task, check what performance they report. You can often get architectural or preprocessing ideas from their work. But beware—published metrics aren't always trustworthy, and they may not be directly comparable to your setup.

Human performance

Can a human do this task? How well would an expert do it? This can help you calibrate expectations.

Experimental replicates

Biological measurements often contain noise due to sampling variability, measurement error, or biological variability itself. One way

to estimate the ceiling for your model's performance is to check how consistent the raw signal is across replicate experiments. If two replicates have a correlation of 0.85, your model is unlikely to exceed that. Don't expect your model to be more consistent than biology itself.

Addressing Poor Model Performance

While there's no universal fix for a model that just isn't performing well, the following strategies can help identify what's wrong and suggest paths forward:

Check data quality

Many model issues are actually data issues. Dive into specific examples—especially ones the model gets wrong—and look for inconsistencies, noise, or labeling errors. If the data is highly domain specific and you're not an expert, ask someone who is.

Run error analysis

Where is the model doing well? Where does it consistently fail? Are there patterns to its mistakes—specific classes, edge cases, or confounding conditions? Systematic errors often point to missing features or broken assumptions.

Add more data

More data can help if the model is underfitting or struggling with rare cases. You can also try synthetic augmentation, bootstrapping from known examples, or generating simulations. Watch how performance scales with dataset size—plateaus may indicate other bottlenecks.

Tune hyperparameters

Some hyperparameters matter more than others—start with learning rate, batch size, model depth, and regularization strength. Use grid or random search over a small range to find better-performing settings.

Try transfer learning

If similar datasets or tasks exist, use pretrained models as a starting point. You can either fine-tune the whole model or freeze its feature extractor and train a smaller model on top. Alternatively, use learned embeddings from a related model as input features.

TIP

As always, if you're stuck, revisit the basics: simplify the model, overfit a single batch, sanity-check your labels, and compare against baseline performance. Many of the strategies that help fix broken models can also clarify why a working model isn't yet a good one.

Final Thoughts

Deep learning in biology is hard. The data is messy, the goals are often open-ended, and training useful models can be finicky. But that's exactly what makes it exciting. With every experiment, you're not just solving a technical challenge—you're helping push the boundaries of how we understand life itself.

The journey won't always be smooth—models will fail in surprising ways, you'll write catastrophic bugs, and the data will contain monumental errors. But if you stay curious, keep things modular, simplify when in doubt, and stay patient, you'll find your way through.

Whether you're building models to decode genomes, predict protein structures, or interpret microscopy images, we hope this book has helped you approach the work more confidently—and maybe even enjoy the process a little more.

Good luck, and keep going!

